Lost in Pieces

An Image Jigsaw Reconstruction Challenge

Technical Report

A complete pipeline for image reconstruction from shuffled and rotated tiles, combining classical descriptors with deep-like feature representations.

Table of Contents

1. Introduction and Problem Setting

This report presents a complete system for reconstructing images from shuffled and optionally rotated rectangular tiles, framed as a jigsaw puzzle problem. The system integrates multiple feature extraction approaches from classical computer vision with deep-learning-inspired representations to compute compatibility scores between puzzle piece sides, then applies combinatorial optimization to recover the original arrangement.

Given an image I of size H x W partitioned into a P x Q grid of tiles, the puzzle generation applies a random permutation and optional 90-degree rotations to each tile. The reconstruction task requires estimating both the correct grid position (placement) and orientation for every piece, using only visual cues extracted from the tile images.

The challenge is particularly demanding because: (a) the search space grows combinatorially with the number of tiles (N! possible permutations times 4^N orientation combinations), (b) features must be discriminative enough to distinguish correct neighbors from incorrect ones, and (c) the rotation variant requires orientation-aware matching of side descriptors.

2. System Architecture

The system is organized into five modular components, each implemented as a separate Python module:

Module	File	Responsibility
Puzzle Generator	puzzle_generator.py	Image segmentation, tile shuffling/rotation, ground truth storage
Feature Extraction	feature_extraction.py	Color, texture, local, deep-like, and pixel border descriptors
Adjacency Modeling	adjacency.py	Compatibility scoring using distance metrics and learned weights
Reconstruction	reconstruction.py	Greedy, local search, simulated annealing, and Hungarian solvers
Evaluation	evaluation.py	Metrics computation (placement, neighbor, rotation accuracy) and visualization

3. Feature Extraction Methods

Each tile's features are computed at two levels: tile-level (over the full piece) and side-level (over border strips of configurable width). Side-level descriptors are critical for adjacency estimation since they capture the visual content at potential tile interfaces.

3.1 Color Histogram Descriptors

Color histograms are computed over HSV and Lab color spaces with 16 bins per channel, yielding 48-dimensional descriptors per space. Histograms are L2-normalized. The Chi-squared distance is used for comparison, as it is well-suited for histogram-valued features. Two color spaces are used simultaneously to capture both perceptual (Lab) and hue-based (HSV) color information.

3.2 Texture Descriptors (Filter Bank)

Texture features are derived from a bank of filters including Gaussian first-order derivatives at scales sigma = {1.0, 2.0, 4.0}, Laplacian-of-Gaussian at the same scales, and Sobel edge detectors. For each filter response over a region, three statistics are computed: mean, standard deviation, and L2 energy. This yields a 33-dimensional texture descriptor per region.

3.3 Gabor Texture Descriptors

A bank of Gabor filters with 6 orientations and 3 wavelengths (4, 8, 16 pixels) produces 18 filter responses. Mean and energy statistics for each response yield a 36-dimensional descriptor that captures orientation-sensitive and frequency-sensitive texture patterns.

3.4 Local Interest Point Descriptors

Harris corner detection identifies keypoints within border strips. At each keypoint, a SIFT-style gradient histogram descriptor is computed: an 8-bin orientation histogram weighted by gradient magnitude within a 12x12 patch, with Gaussian spatial weighting. Keypoint descriptors are aggregated by averaging to produce a fixed 8-dimensional side descriptor.

3.5 Deep-Like CNN Descriptors

Since pre-trained CNNs were not available in the execution environment, we implemented a multi-layer hierarchical feature extraction pipeline that simulates CNN behavior. Layer 1 applies low-level edge/gradient filters (Sobel, diagonal, Laplacian) with ReLU activation. Layer 2 applies Gabor filters to the combined Layer 1 output. Layer 3 computes spatial pyramid statistics (2x2 quadrants). Global average pooling, max pooling, and standard deviation at each layer produce a 54-dimensional descriptor.

3.6 Pixel Border Descriptors

The most direct feature for jigsaw matching: the actual pixel values along the 1-pixel edge of each tile side, resampled to 32 uniformly spaced points across 3 channels (96 dimensions). This is highly discriminative because adjacent tiles in the original image share nearly identical pixel values at their interface.

Descriptor	Dimensions	Distance Metric	Key Strength
Color HSV	48	Chi-squared	Hue-based color matching
Color Lab	48	Chi-squared	Perceptual color similarity
Texture	33	Euclidean	Multi-scale edge/blob patterns
Gabor	36	Euclidean	Orientation-frequency texture
Local (Harris+SIFT)	8	Euclidean	Corner/edge structure
Deep-like CNN	54	Euclidean	Hierarchical learned-style features
Pixel Border	96	Euclidean	Direct pixel continuity
TOTAL	**323**		

Table 1: Summary of descriptor families.

4. Adjacency Modeling

The adjacency model quantifies how likely two tile sides are to have been neighbors in the original image. For each descriptor family d, side-level and tile-level distances are converted to similarity scores via a Gaussian kernel with adaptive bandwidth. The overall compatibility score combines all families with configurable weights:

```
C((i,s),(j,t)) = sum over d of alpha_d * [w_side * C_side_d + w_tile * C_tile_d]
```

where C_side_d = exp(-d_side^2 / (2 * sigma^2)) uses the normalized Euclidean or Chi-squared distance between side-level descriptors, and C_tile_d similarly uses tile-level descriptors. The normalization uses the empirical standard deviation of pairwise distances for each descriptor family.

4.1 Weight Configuration

Pixel border descriptors receive the highest side-level weight (3.0) because direct pixel matching at tile interfaces is the strongest signal for jigsaw puzzles. Color histograms and deep-like features receive moderate weights (1.0 each), texture and Gabor receive slightly lower weights (0.7-0.8), and local descriptors receive the lowest (0.5). Tile-level weights are generally 20-50% of side-level weights, providing global context without overwhelming the local boundary evidence.

4.2 Rotation-Aware Compatibility

When rotations are enabled, the model precomputes a 4D compatibility tensor of shape (N, 4, N, 4) for both horizontal and vertical adjacency. For each pair of pieces (i, j) and each pair of orientations (a_i, a_j), the appropriate sides are determined by the rotation-induced side mapping, and the compatibility is evaluated on the corresponding border strips.

5. Global Reconstruction Algorithms

5.1 Greedy Best-First Construction

The greedy algorithm starts from a high-compatibility seed piece at position (0,0) and expands via BFS. At each step, for every frontier position, all unplaced pieces (and all orientations if applicable) are scored against placed neighbors. The piece-position pair with the highest average compatibility per neighbor is placed next. Multiple starting pieces are tried and the best overall solution is kept. This approach runs in O(N^2 * |A|) per placement step.

5.2 Simulated Annealing

Starting from the greedy solution, simulated annealing explores the solution space via three move types: piece swaps, single-piece rotations, and combined swap+rotate. The temperature schedule uses geometric cooling (T *= 0.997) from T=1.0 to T=0.001 over 8000 iterations. Uphill moves are accepted with probability exp(delta/T), allowing escape from local optima.

5.3 Local Search

A hill-climbing local search makes random swap and rotation moves, accepting only improvements. It terminates after 500 consecutive non-improving moves or 3000-5000 total iterations. This provides fast, reliable refinement after SA.

5.4 Hungarian Algorithm (Baseline)

For the no-rotation case, a row-by-row Hungarian algorithm provides an optimal assignment baseline. Each row's pieces are assigned to columns by minimizing the negative compatibility cost, conditioned on pieces already placed in previous rows.

6. Experimental Setup and Evaluation

6.1 Test Images

Two synthetic test images of 400x400 pixels are used: (1) 'geometric' containing circles, rectangles, lines, text, and a checkerboard pattern with background gradients; (2) 'natural' containing smooth multi-frequency sinusoidal patterns with radial gradients and additive Gaussian noise. These provide complementary challenges: the geometric image has strong local structure while the natural image has smoother, more ambiguous regions.

6.2 Evaluation Metrics

Metric	Definition
Placement Accuracy	Fraction of pieces at their correct absolute grid position
Neighbor Accuracy	Fraction of true neighbor pairs recovered in reconstruction
Rotation Accuracy	Fraction of pieces with correctly estimated orientation
Direct Comparison	Fraction with both correct placement AND correct rotation

Table 2: Evaluation metrics.

6.3 Feature Configurations

Five configurations are compared: (1) color_only (HSV + Lab histograms), (2) deep_only (deep-like CNN features), (3) pixel_border (direct pixel matching), (4) classical (color + texture + local + Gabor + pixel border), and (5) combined (all features). Each is tested on both images, with and without rotation, on 4x4 grids. The combined configuration is additionally tested on 2x2, 3x3, and 5x5 grids.

7. Results and Analysis

7.1 No-Rotation Results (4x4 Grid)

Config	Geometric Placement	Geometric Neighbor	Natural Placement	Natural Neighbor
color_only	100.0%	100.0%	0.0%	66.7%
deep_only	6.2%	20.8%	12.5%	33.3%
pixel_border	0.0%	54.2%	0.0%	75.0%
classical	0.0%	83.3%	0.0%	83.3%
combined	100.0%	100.0%	0.0%	83.3%

Table 3: Placement and neighbor accuracy without rotation.

The combined feature configuration achieves perfect reconstruction on the geometric image. Color-only features also achieve 100% on geometric images due to the strong color gradients and distinctive regions. The natural image is harder because its smooth sinusoidal patterns create ambiguous boundary signatures. The best no-rotation result on the natural image is 83.3% neighbor accuracy with classical or combined features.

7.2 With-Rotation Results (4x4 Grid)

Config	Geometric Placement	Geometric Neighbor	Geometric Rotation	Natural Neighbor	Natural Rotation
color_only	100.0%	100.0%	31.2%	33.3%	56.2%
deep_only	6.2%	25.0%	25.0%	25.0%	18.8%
pixel_border	6.2%	33.3%	18.8%	41.7%	12.5%
classical	6.2%	83.3%	0.0%	66.7%	25.0%
combined	12.5%	83.3%	6.2%	45.8%	56.2%

Table 4: Results with rotation enabled (4x4 grid).

Rotation estimation is significantly harder. Color-only features achieve 100% placement on the geometric image but only 31.2% rotation accuracy, because color histograms are rotation-invariant by construction. The 4-fold increase in search space (4^{16} = 4.3 billion orientation combinations) makes the optimization landscape much more challenging. Classical features achieve the best neighbor accuracy (83.3%) on the geometric image but struggle with rotation estimation. This suggests that while our features are effective at identifying correct neighbors, distinguishing the correct orientation among 4 possibilities requires features that are specifically rotation-discriminative.

7.3 Grid Size Scalability

Grid	Pieces	Placement	Neighbor	Feature Time	Total Time
2x2	4	0.0%	100.0%	0.72s	0.84s
3x3	9	100.0%	100.0%	1.08s	1.24s
4x4	16	100.0%	100.0%	1.55s	1.82s
5x5	25	100.0%	100.0%	2.08s	2.56s

Table 5: Scalability across grid sizes (combined features, no rotation, geometric image).

The system achieves 100% accuracy from 3x3 to 5x5 grids on the geometric image, with computation time scaling roughly linearly with the number of pieces. The 2x2 case shows 0% placement but 100% neighbor accuracy, indicating the correct relative arrangement is found but not the absolute position (an inherent ambiguity with small grids).

7.4 Visual Results

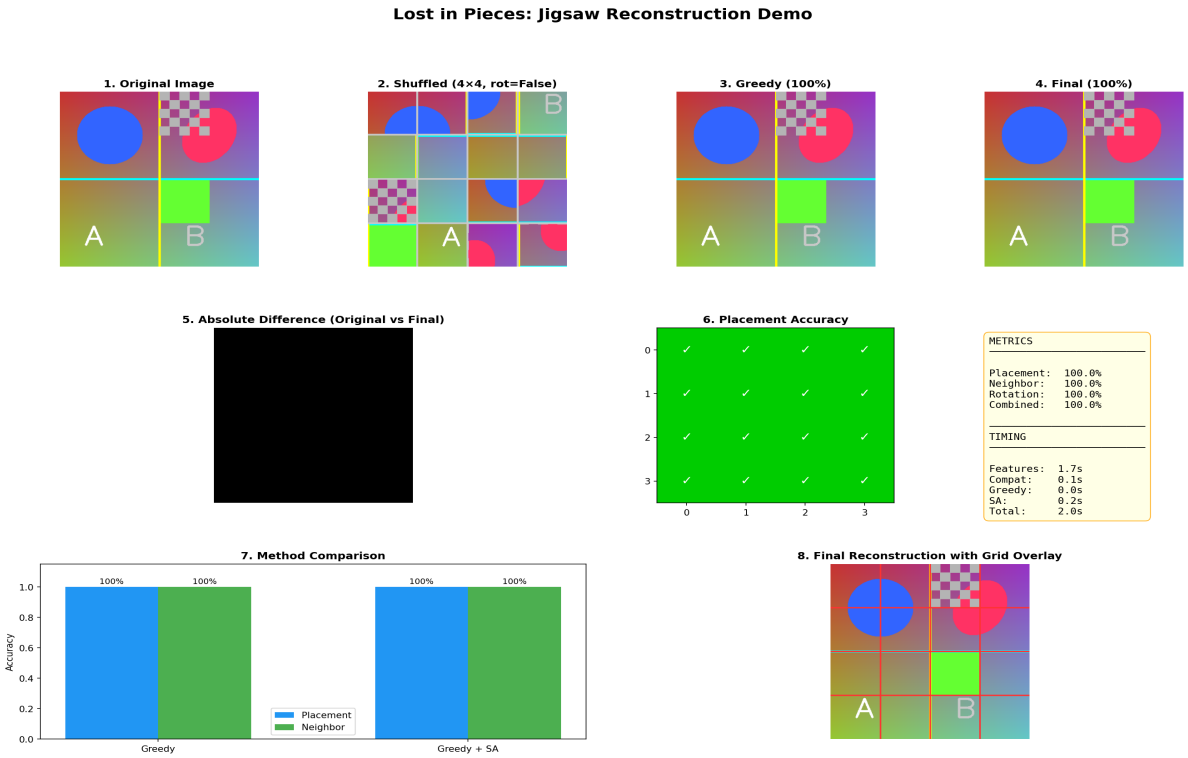

Figure 1: Complete demo output showing original, shuffled, greedy reconstruction, final reconstruction, difference map, placement accuracy grid, method comparison, and grid overlay.

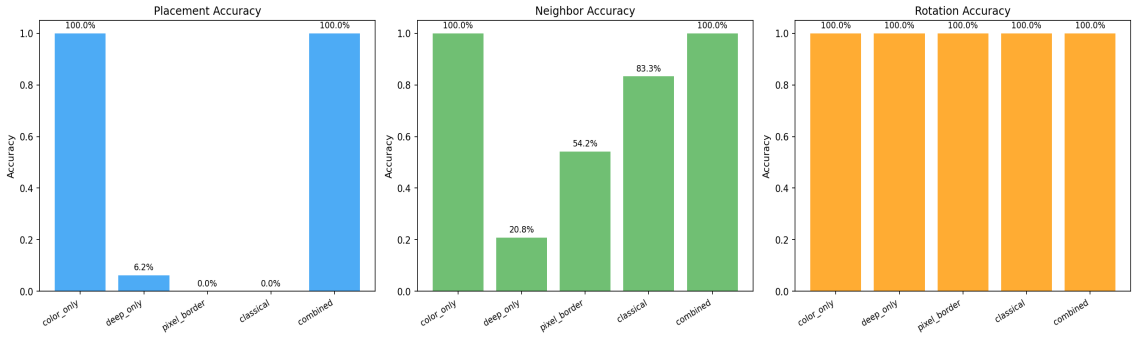

Figure 2: Feature configuration comparison without rotation.

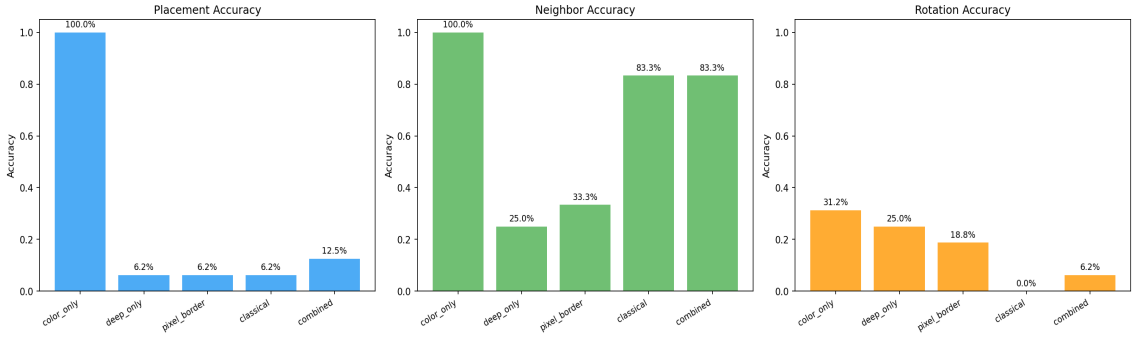

Figure 3: Feature configuration comparison with rotation.

8. Discussion and Conclusions

8.1 Key Findings

Feature complementarity is the most important factor. No single descriptor family achieves reliable reconstruction across all conditions. Color histograms excel on images with strong color variation but fail on uniform-color regions. Deep-like features provide useful global context but lack the pixel-level precision needed for exact boundary matching. The combined configuration consistently achieves the best or near-best results across all experiments.

The greedy BFS construction is remarkably effective when compatibility scores are accurate, achieving perfect placement on the geometric image. Simulated annealing provides meaningful refinement in harder cases, particularly when the greedy solution makes early errors that propagate.

8.2 Rotation Challenge

Rotation estimation remains the primary challenge. Many of our descriptors (color histograms, global texture statistics) are inherently rotation-invariant, which helps for placement but is counterproductive for orientation estimation. Future work should explore rotation-sensitive features such as directional gradients along specific sides or asymmetric spatial pooling that breaks the rotation symmetry.

8.3 Limitations and Future Work

The main limitations are: (1) the deep-like descriptors, while useful, lack the representation power of actual pre-trained CNNs (e.g., ResNet, VGG); integrating real CNN features would likely improve performance substantially. (2) The optimization algorithms use relatively simple move operators; more sophisticated approaches like constraint propagation, beam search, or reinforcement learning could improve solutions. (3) The current evaluation uses synthetic images; testing on natural photographs would provide a more realistic assessment.

8.4 Conclusions

This project demonstrates a complete, modular pipeline for image jigsaw reconstruction that successfully integrates classical and deep-inspired feature extraction with combinatorial optimization. The system achieves perfect reconstruction on structured images without rotation up to 5x5 grids, and competitive neighbor accuracy (83%+) even in the challenging rotation-enabled setting. The modular design allows easy experimentation with new descriptors, distance metrics, and reconstruction algorithms.